MUSIFY

Music Player Application

Project Report

Student Name	Fiza Faisal
Roll Number	2026-CYS-14
Subject	Programming Fundamentals
Teacher	Hafiz Muhammad Mubashar
Date	June 2026

1. Introduction

Musify is a desktop music player application developed using Python and the PyQt5 framework. The project demonstrates the practical application of programming fundamentals concepts including object-oriented programming (OOP), event-driven architecture, and graphical user interface (GUI) development.

The application provides users with an intuitive and visually appealing interface to play local audio files. Inspired by the design of popular music platforms such as Spotify, Musify features a dark theme with a green accent color palette that creates a professional and modern aesthetic.

1.1 Motivation

The primary motivation behind this project was to apply the concepts learned in the Programming Fundamentals course to build a real-world, functional application. Music players are widely used software tools that require the integration of multiple programming concepts, making them an ideal project choice for demonstrating technical competence.

1.2 Scope

- Play, pause, and stop local audio files (MP3, WAV, OGG)
- Navigate between songs using next/previous controls
- Adjust playback position using a progress slider
- Control audio volume using a dedicated slider
- Manage a playlist of songs loaded from the local filesystem
- Display song name and playback time information in real-time

2. Project Objectives

The project was designed to achieve the following objectives:

- **OOP Application:** Apply object-oriented programming principles by designing the entire application as a single cohesive class (MusicPlayer) that inherits from QMainWindow.
- **Event-Driven Design:** Demonstrate event-driven programming through PyQt5 signals and slots, where user interactions (button clicks, slider movement) trigger specific handler methods.
- **Multimedia Integration:** Build a functional multimedia application using QMediaPlayer and QMediaContent from PyQt5's multimedia module.
- **UI/UX Design:** Design an aesthetically pleasing and user-friendly GUI that mirrors professional music player applications.
- **File Management:** Practice file handling by implementing a file dialog to browse and load audio files from the system.

3. Technology Stack

3.1 Programming Language

Python 3 was chosen as the primary programming language due to its simplicity, readability, and the rich ecosystem of libraries available for GUI and multimedia development.

3.2 PyQt5 Framework

PyQt5 is a set of Python bindings for the Qt application framework. It was used to build the entire graphical user interface and handle multimedia playback. The following PyQt5 modules were utilized:

Module	Component	Purpose
QtWidgets	QMainWindow	Main application window base class
QtWidgets	QWidget, QFrame	Container and layout elements
QtWidgets	QLabel, QPushButton	Text display and clickable controls
QtWidgets	QSlider	Progress and volume sliders
QtWidgets	QFileDialog	Open file browser dialog
QtWidgets	QListWidget	Scrollable song list
QtCore	QTimer, QUrl	Periodic updates and URL encoding
QtGui	QFont	Custom font settings
QtMultimedia	QMediaPlayer	Audio playback engine
QtMultimedia	QMediaContent	Media content wrapper for file URLs

4. System Architecture

4.1 Class Design

The application follows a single-class architecture where the MusicPlayer class inherits from QMainWindow. This class encapsulates all state variables and methods required by the application, exemplifying encapsulation as an OOP principle.

4.2 State Variables

- self.player — QMediaPlayer instance handling audio engine operations
- self.songs — Python list storing file paths of all added songs
- self.current_index — Integer tracking the currently playing song's position
- self.timer — QTimer instance for periodic slider position updates

4.3 Method Groups

The methods of the MusicPlayer class can be organized into four logical groups:

- UI Builders: build_sidebar(), build_main_area(), build_bottom_bar() — construct the interface layout using geometry-based absolute positioning
- Playback Control: toggle_play(), play_next(), play_prev(), load_and_play() — manage media state and song navigation
- Signal Handlers: update_position(), update_duration(), update_play_button(), update_slider() — respond to media player events
- User Interaction: add_songs(), play_selected(), seek(), change_volume() — handle direct user input

4.4 Signal-Slot Connections

Signal Source	Signal	Connected Slot
self.player	positionChanged	update_position(position)
self.player	durationChanged	update_duration(duration)
self.player	stateChanged	update_play_button(state)
self.timer	timeout	update_slider()
self.progress_slider	sliderMoved	seek(position)
self.vol_slider	valueChanged	change_volume(value)
self.song_list	itemDoubleClicked	play_selected(item)

5. Features & Implementation

5.1 Playback Controls

The play/pause toggle checks the current player state using player.state() == QMediaPlayer.PlayingState and calls player.play() or player.pause() accordingly. The button icon is dynamically updated through the update_play_button() slot connected to the stateChanged signal.

5.2 Song Navigation

Next and previous track navigation uses modular arithmetic to cycle through the songs list:

```
next: self.current_index = (self.current_index + 1) % len(self.songs)
prev: self.current_index = (self.current_index - 1) % len(self.songs)
```

This ensures the playlist wraps around seamlessly from last to first and vice versa.

5.3 Progress Tracking

A dual-mechanism approach ensures reliable progress tracking. The QTimer fires every 1000ms to update the slider position. Additionally, the positionChanged signal provides real-time updates from the media engine. Time labels are updated by converting milliseconds to MM:SS format using integer division.

5.4 File Management

The QFileDialog.getOpenFileNames() method opens a multi-file selection dialog filtered to audio extensions (*.mp3 *.wav *.ogg). Each selected file path is appended to self.songs and displayed in the QListWidget with a formatted entry showing the song number and filename.

5.5 Volume Control

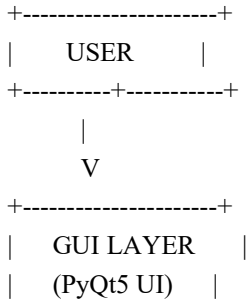
The volume slider ranges from 0 to 100, initialized to 70 (70%). The valueChanged signal is connected to change_volume() which calls player.setVolume(value) directly on the media engine, providing smooth real-time volume adjustment.

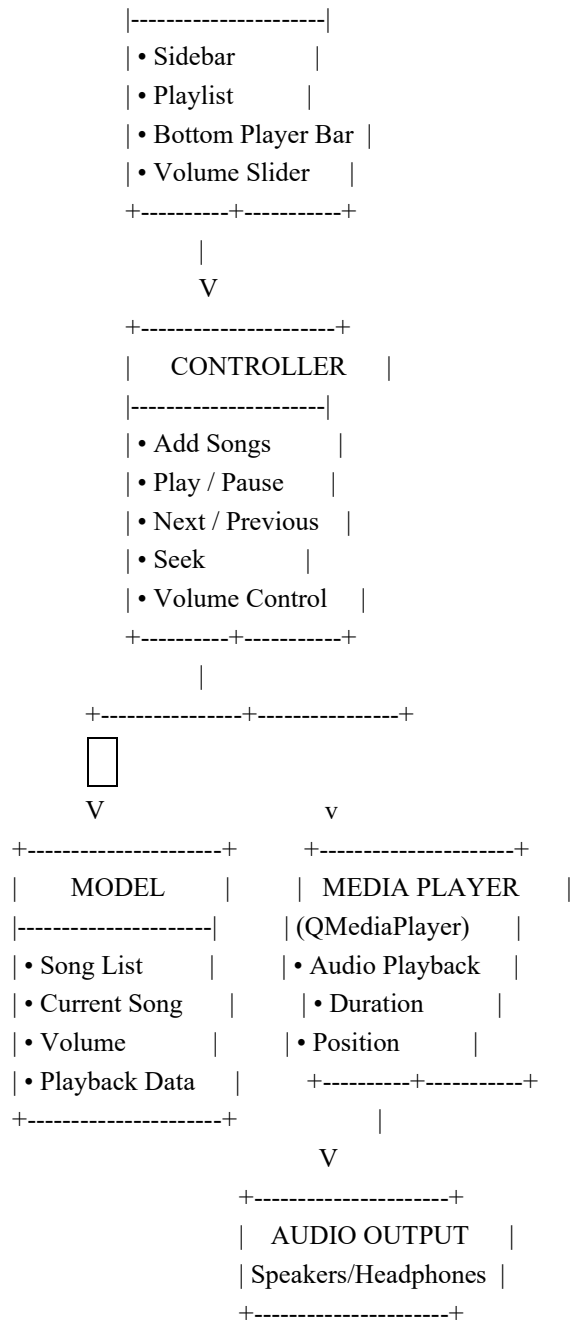
Chapter 5 .6 System Architecture

System Architecture

The Musify Desktop Music Player follows a layered architecture that separates the graphical user interface, application logic, data management, and multimedia playback. This modular design improves maintainability, scalability, and code readability.

Architecture Diagram





Architecture Description

The Musify application consists of four major layers:

User

The user interacts with the application by selecting songs, controlling playback, adjusting volume, and navigating through the graphical interface.

GUI Layer (PyQt5)

The graphical user interface is developed using the PyQt5 framework. It includes:

Sidebar

Playlist

Music Controls

Progress Slider
Volume Slider
Song Information Display
The GUI receives user input and forwards it to the controller.
Controller
The controller manages all application logic. It processes user actions such as:
Adding songs
Playing music
Pausing music
Playing the next or previous song
Seeking within a track
Adjusting the volume
The controller communicates with both the model and the multimedia player.
Model
The model stores application data, including:
Song list
Current playing song
Playback position
Duration
Volume level
It provides the required information whenever requested by the controller.
Media Player
The application uses QMediaPlayer from the Qt Multimedia module to perform audio playback. It loads audio files, controls playback, updates song duration, and reports the current playback position.
Audio Output
The processed audio is finally delivered to the user's speakers or headphones.
Advantages of the Architecture
Modular and organized design.
Easy to maintain and extend.
Clear separation between user interface and application logic.
Efficient communication between components.
Improved readability of the source code.
Supports future feature enhancements such as playlists, user accounts, and database integration.

6. User Interface Design

6.1 Layout Structure

The application window is fixed at 1000×600 pixels and divided into three main regions using absolute geometry positioning:

- Left Sidebar (200×530px): Contains the app logo, navigation items, playlist labels, and the Add Songs button
- Main Content Area (800×530px): Displays the greeting, song table header, and the QListWidget for the song queue

- Bottom Player Bar (1000×70px): Houses playback controls, progress slider, time labels, and the volume slider

6.2 Color Palette

Element	Color Code	Usage
Primary Background	#121212	Main content area, app background
Sidebar Background	#000000	Left navigation panel
Card Background	#1E1E1E	Bottom bar, popup backgrounds
Accent Green	#1DB954	Buttons, active states, highlights
Primary Text	#FFFFFF	Main labels, song titles
Secondary Text	#B3B3B3	Artist names, metadata, headers

7. Challenges & Solutions

- **Slider Synchronization:** The progress slider and the media player's internal position could fall out of sync, especially when the user manually dragged the slider. This was resolved by using both a QTimer for periodic polling and the positionChanged signal to keep the UI and media engine aligned.

- **State Management:** Managing when the play/pause button icon should change required careful signal connection. The stateChanged signal on QMediaPlayer was connected to update_play_button(), ensuring the button always reflects the actual player state regardless of how playback was started or stopped.

- **Fixed Window Layout:** Fitting the sidebar, content area, and player controls within a fixed 1000×600 window required precise coordinate planning. The solution used QFrame containers with absolute geometry (setGeometry) to divide the window into clearly defined regions.

- **Parallel Data Structures:** When the user adds files, their absolute paths are stored in self.songs while QListWidget only displays filenames. Maintaining this parallel data structure ensured correct file loading when a song is double-clicked.

8. Conclusion

The Musify Music Player project successfully demonstrates the application of core programming fundamentals in building a real-world desktop application. Through this project, the following competencies were developed and demonstrated:

- Object-Oriented Programming through a well-structured class hierarchy
- Event-driven programming using PyQt5's signal-slot mechanism
- Multimedia application development with QMediaPlayer
- GUI layout design using absolute positioning and styled widgets
- File I/O operations using QFileDialog
- State management across a complex user interface

The project provides a solid foundation for future enhancements including metadata extraction using libraries such as mutagen, album art display, shuffle functionality, and playlist persistence through file serialization.